Torn Faction Operations Suite - Comprehensive Implementation Plan

Goal Description

Build an end-to-end, high-performance, Terms-of-Service-compliant operations suite for Torn City factions. The suite solves the 7 key challenges of competitive faction management:

1. **Faction Discord War Bot**: Real-time Ranked War (RW) and chain watchdog (alerting at <120s, <60s, <30s), automated enemy hospital callouts, and enemy activity/sleep heatmap analysis.
2. **War Payout & Fair-Share Calculator**: Ingestion of `rankedwarreport` and `attacksfull`, customizable fair-share reward formulas (base hit, respect, fair fight stat multiplier, clutch saves), and 1-click JSON/CSV export for in-game payout helpers.
3. **Organized Crime (OC) 2.0 Team Optimizer**: Skill scoring across Crimes 2.0 proficiencies, automated optimal slot assignment, and Discord nudge alerts for stalled invites holding up faction crime timers.
4. **Custom Tampermonkey Userscripts**: Browser extensions providing in-game war target filters, faction armory restock warnings, and energy/nerve/travel desktop notifiers.
5. **Faction Banking & Vault Ledger**: Automated deposit tracking via `fundsnews`, Discord `/withdraw` command & web withdrawal requests, itemized banker fulfillment queue with 1-click fast-copy batch strings for Torn vault transfers.
6. **Baldr's Leveling & Chain Target Directory**: Interactive target finder for high-XP / low-stat targets with live status (hospital, traveling, okay) and direct attack links.
7. **Faction & Member Kanban Todo Board**: Shared faction war/training goals and individual member task lists integrated with war and OC milestones.

All features strictly observe Torn's 100 requests/minute rolling API limit using an intelligent rate-limiter and key-pooling architecture, and adhere strictly to Torn ToS (zero automated in-game actions; human clicks only).

Architecture & System Topology

User Review Required

> IMPORTANT
>
> **Hosting Split**: The Web Dashboard will be deployed on Vercel, connecting to Supabase PostgreSQL via connection pooling (Transaction / Session pooler). The Discord Bot and high-frequency background polling daemons (Chain & Hospital watchdog) require a long-running Node.js process (e.g., Docker container, Railway, Render, Fly.io, or dedicated VPS) because Vercel serverless functions cannot maintain persistent WebSocket gateway connections or sub-second polling loops.

> NOTE
>
> **API Key Pooling & Security**: Faction endpoints (wars, crimes, vault fundsnews) require a Leader/Co-Leader API key with Access Authority (AA). Personal keys contributed to the pool will be securely encrypted at rest (AES-256-GCM) and used solely for public endpoints (such as checking enemy hospital status and member profiles) to multiply the faction's available rate limit beyond 100 req/min.

Open Questions & Configuration Options

- **Discord Bot Token & Application ID**: Will be supplied via environment variables (`DISCORD_BOT_TOKEN`, `DISCORD_CLIENT_ID`).
- **Database URL**: Will support standard Supabase connection strings (`DATABASE_URL` for pooled connections and `DIRECT_URL` for Prisma migrations).

Proposed Changes & File Layout

We will structure the project as a Turborepo monorepo with pnpm workspaces:

```
torn-suite/
├── apps/
│   ├── web/                    # Next.js 15 Web Dashboard (App Router, Tailwind CSS,
shadcn/ui)
│   └── bot/                    # Discord.js v14 Bot & Realtime Watchdog Daemons
├── packages/
│   ├── db/                     # Prisma schema, migrations, and shared database
client
│   ├── torn-api/               # Torn v2 API client, rate limiter & token bucket key
pool
│   └── shared/                 # Shared types, Zod schemas, payout & OC formula
engines
├── userscripts/                # Standalone Tampermonkey userscripts
│   ├── torn-target-filters.user.js
│   ├── torn-armory-warning.user.js
│   └── torn-energy-travel-notifier.user.js
├── turbo.json
├── package.json
└── pnpm-workspace.yaml
```

Component 1: `packages/db` (Persistence & Database Models)

Houses the Prisma schema configured for PostgreSQL (Supabase compatible).

[NEW] `packages/db/prisma/schema.prisma`

Key models include:

- `User`: Discord ID, username, avatar, Torn ID, Torn username, role (`MEMBER`, `OC_LEAD`, `BANKER`, `CO_LEADER`, `LEADER`).
- `ApiKeyPool`: Encrypted key string, owner Torn ID, key type (`FACTION`, `MEMBER`), calls last minute, active status.
- `FactionConfig`: Faction ID, name, discordGuildId, chainChannelId, hospChannelId, warChannelId, bankChannelId, alertRoles.
- `War`: Torn war ID, opponentFactionId, opponentName, startTimestamp, endTimestamp, status, reportCache.
- `AttackCache`: Attack code, attackerId, defenderId, result, respect, timestamp, modifier, isClutch.
- `EnemyWatch`: Torn ID, name, factionId, hospitalUntil, lastKnownStatus, priorityScore, sleepPatternJson.
- `BankAccount`: Member Torn ID, balance, totalDeposited, totalWithdrawn, updatedAt.

- `BankTransaction`: Transaction ID, memberTornId, type (`DEPOSIT`, `WITHDRAWAL`, `WAR_PAYOUT`, `ADJUSTMENT`), amount, tornLogId, status, processedBy, createdAt.
- `WithdrawalRequest`: Request ID, memberTornId, amount, reason, status (`PENDING`, `APPROVED`, `FULFILLED`, `REJECTED`), requestedAt, fulfilledAt.
- `OrganizedCrime`: Crime ID, name, slotCount, minSkillRequired, status (`PLANNING`, `READY`, `EXECUTED`).
- `CrimeAssignment`: Crime ID, memberTornId, slotIndex, invitedAt, acceptedAt, reminderCount.
- `LevelingTarget`: Target Torn ID, name, level, estimatedStats, lastUpdated, notes, difficultyTier.
- `TodoTask`: Task ID, title, description, assignedToId, createdById, status (`TODO`, `IN_PROGRESS`, `DONE`), priority, dueDate, factionWide.

Component 2: `packages/torn-api` (Torn v2 API SDK & Key Pooling Engine)

[NEW] `packages/torn-api/src/rate-limiter.ts`

- Implements a rolling-window Token Bucket rate limiter (max 100 requests per rolling 60 seconds per key).
- Multi-key router: Distributes incoming requests across available keys in the pool based on key permissions (Public vs Faction).
- Automatic back-off and retry logic when encountering HTTP 429 or Torn API error code 5 (Too many requests).

[NEW] `packages/torn-api/src/client.ts`

- Full TypeScript client wrapping Torn API v2 endpoints:
  - `/faction/basic`, `/faction/rankedwars`, `/faction/rankedwarreport`, `/faction/fundsnews`, `/faction/crimes`, `/faction/armory`
  - `/user/basic`, `/user/profile`, `/user/attacksfull`, `/user/crimes`
  - `/torn/rankedwars`, `/torn/items`
- Structured return types with Zod validation.

Component 3: `packages/shared` (Business Logic & Formula Engines)

[NEW] `packages/shared/src/payout-calculator.ts`

- Ingests attacks list and war results.
- Configurable Fair-Share formula: $\text{Payout}(u) = \text{TotalPool} \times \frac{\text{Score}(u)}{\sum \text{Score}}$
  Payout(u) = TotalPool × ∑ ScoreScore(u)

$$\text{Score}(u) = (\text{Hits} \times W_{\text{hit}}) + (\text{Respect} \times W_{\text{resp}}) + (\text{Assists} \times W_{\text{assist}}) + (\text{ClutchSaves} \times W_{\text{clutch}}) - (\text{Losses} \times W_{\text{loss}})$$
Score(u) = (Hits × Whit) + (Respect × Wresp) + (Assists × Wassist) + (ClutchSaves × Wclutch) − (Losses × Wloss)

- Exporter generating:
    1. JSON format compatible with Torn Payout Userscripts.
    2. Clean CSV export for spreadsheets.

[NEW] `packages/shared/src/oc-optimizer.ts`

- Skill scoring matrix matching member stats to Crime 2.0 requirements (Burglary, Cybercrime, Illicit Services, etc.).
- Bipartite matching algorithm finding the configuration that maximizes team success chance while leaving available backup members.

Component 4: `apps/bot` (Discord Bot & Real-time Watchdogs)

[NEW] `apps/bot/src/index.ts`

- Boots Discord.js client, registers slash commands:
    - `/setup`: Configure channels (`#chain-watchdog`, `#hospital-callouts`, `#war-announcements`, `#banking-ledger`) and ping roles.
    - `/chain`: View current chain count, time remaining, and recent hitters.
    - `/hospital`: List active enemy targets out of hospital with direct attack links.
    - `/withdraw <amount> [reason]`: Submit vault withdrawal request.
    - `/balance`: Check personal faction bank balance.
    - `/targets [min_level] [max_level]`: Pull quick leveling / chain targets.
    - `/oc status`: View current OC team status and pending unaccepted invites.
    - `/todo [list|add|complete]`: Manage personal and faction tasks.

[NEW] `apps/bot/src/watchdogs/chain-watchdog.ts`

- High-frequency polling during active wars (every 3–5 seconds).
- Escalating alerts:
    - 🟡 Warning at 120s remaining.
    - 🟠 High Alert at 60s remaining.
    - 🔴 Emergency Ping at 30s remaining (pings configured Chain Alert role).
- Clutch hit recognition: Detects hits landed with <30s remaining and flags them in the database for payout bonuses.

[NEW] `apps/bot/src/watchdogs/hospital-watchdog.ts`

- Monitors opposing faction roster during Ranked Wars.
- Tracks exact hospital release timestamps.
- When a high-value enemy target hospital timer reaches 0, immediately dispatches a Discord callout embed with target link, level, and estimated battle stats.

[NEW] `apps/bot/src/watchdogs/oc-nudge-watchdog.ts`

- Runs hourly; queries active OC teams with pending invitations.
- Finds members who haven't accepted within X hours and sends a targeted Discord ping / DM.

[NEW] `apps/bot/src/watchdogs/fundsnews-poller.ts`

- Polls `/faction/fundsnews` every 60 seconds.
- Detects new vault deposit logs, matches depositor Torn ID, and updates the member's `BankAccount` balance in the database automatically.

Component 5: `apps/web` (Next.js 15 Web Dashboard)

Modern dark-mode UI styled with Tailwind CSS and Radix/shadcn components.

[NEW] Pages & Routes:

- `/` : Overview dashboard (live war status, chain countdown widget, faction balance, quick shortcuts).
- `/war` : Ranked War Command Center (live score differential, hospital monitor, enemy sleep/activity heatmap visualization).
- `/payout` : War Payout & Fair-Share Calculator:
  - Fetch war report by ID.
  - Interactive formula slider weights (base pay, respect weight, clutch bonus).
  - Payout breakdown table with 1-click JSON copy and CSV download.
- `/oc` : OC 2.0 Team Optimizer:
  - Team roster overview, skill match scores, automated team auto-fill recommendations, stalled invite indicators.
- `/banking` : Faction Vault & Banking Ledger:
  - Member balances table.
  - Recent deposit ledger (synced from `fundsnews`).

    - Pending withdrawal requests with batch select and **"Copy Torn Vault Transfer String"** button to execute payouts in Torn in seconds.

- `/targets`: Baldr's Leveling Targets Directory:
    - Filterable by level range, difficulty tier, online/offline status, hospital state.
    - One-click attack buttons linking directly to `torn.com/loader.php?sid=attack&user2ID=...`.
- `/todo`: Faction & Member Kanban Board:
    - Columns: Backlog, Todo, In Progress, Completed.
    - Filter by personal tasks vs faction-assigned goals.
- `/settings`: API Key management (Faction master key + voluntary member key pool), Discord webhook test, role permissions.

Component 6: `userscripts` (Tampermonkey Browser Extensions)

[NEW] `userscripts/torn-target-filters.user.js`

- Injects a responsive control panel directly onto the Torn Ranked War page (`torn.com/factions.php?step=your#/war/rank`).
- Features:
    - Filter out enemy targets currently in hospital, jail, or abroad.
    - Filter targets above a designated level or estimated battle stat threshold.
    - Direct quick-attack button injection next to each valid target.

[NEW] `userscripts/torn-armory-warning.user.js`

- Monitors faction armory medical supplies (Morphine, First Aid Kits, Blood Bags).
- Renders an alert banner directly on the faction armory page if quantities drop below customizable threshold levels.

[NEW] `userscripts/torn-energy-travel-notifier.user.js`

- Monitors player energy, nerve, and travel timer using native browser `Notification` API.
- Alerts user when energy reaches 100/150 or when airplane lands at destination.

Verification Plan

Automated Tests

1. **API Client & Rate Limiter**:
   - `pnpm --filter @torn/api test`: Test token-bucket rate limiter under simulated concurrent load (ensuring no more than 100 req/min is emitted per key).
   - Test key pool rotation and back-off on simulated 429 response.
2. **Payout Formula Engine**:
   - `pnpm --filter @torn/shared test`: Validate fair-share formula calculations against mock war logs with edge cases (zero hits, negative respect, ties, clutch saves).
3. **Database Migrations & Queries**:
   - `pnpm --filter @torn/db test`: Verify Prisma schema relationships, ledger balance constraints, and transaction atomicity.

Manual Verification

1. **Discord Bot Interactions**:
   - Run `/setup` and verify bot assigns and remembers channels.
   - Run `/withdraw` and observe pending queue generation.
   - Run `/targets` and verify live target list rendering.
2. **Dashboard UI**:
   - Verify dark-mode interface responsive behavior on desktop and mobile.
   - Test Payout Calculator with sample war dataset and verify JSON/CSV exports.
   - Test Banking Ledger with mock deposits and withdrawals.
3. **Userscripts**:
   - Load userscripts into Tampermonkey in browser and confirm DOM injection on relevant Torn URL routes.